

P2876R2

Proposal to extend `std::simd` with more constructors and accessors

Published Proposal, 2025-02-14

This version:

<https://isocpp.org/files/papers/P2876R2.html>

Authors:

[Daniel Towner](#) (Intel)

[Matthias Kretz](#) (GSI)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to provide additional constructors and accessors for `std::simd_mask`

Table of Contents

1 Motivation

2 Bitset

2.1 Implementation experience

3 Conversion to and from integral value bit representations

3.1 Building a mask from a compact bit representation

3.2 Compact bit accessor

3.3 Implementation experience

4 Initialising from a list

5 Wording

5.1 Modify [[simd.mask.overview](#)]

5.2 Modify [[simd.mask.ctor](#)]

5.3 Modify [[simd.mask.conv](#)]

5.4 Modify [[simd.mask.namedconv](#)]

6 Revision History

References

Informative References

§ 1. Motivation

The [[P1928R3](#)] proposal outlines a number of constructors and accessors which can be used to move data in and out of a `simd` object. However, there are a number of other data types in the standard C++ libraries which provide a form of data-parallel value storage, such as `std::bitset` and types which implement the `std::contiguous_range` concept such as `std::span`. It is desirable to be able to easily convert those to and from `std::simd` values too. In this paper we shall examine the benefits of providing constructors for building `simd` or `simd_mask` values from other types, and for adding accessors which allow them to be converted back into other types from their equivalent SIMD values. Some of these proposals were briefly outlined in [[P2638R0](#)], and some proposals are new.

§ 2. Bitset

`std::bitset` has many characteristics in common with a `simd_mask` value and it is useful to be able to use a `std::bitset` value as a mask and vice versa. There is no easy way for a programmer to concisely and efficiently achieve these interchange conversions so we propose that a constructor and an accessor are provided in `std::simd`. Firstly, a constructor can be provided for a `simd_mask`:

```
constexpr basic_simd_mask(const bitset<size()>& b) noexcept;
```

This constructor allows a bitset with the same number of elements as a SIMD value to be used to build a `simd_mask` for that value. Each element in the constructor mask has the same boolean value as the

respective element from the incoming bitset. It is not marked as explicit, so a bitset can be conveniently used anywhere that a `simd_mask` could be.

A `simd_mask` can be converted into an equivalent `bitset` using a conversion operator:

```
constexpr basic_simd_mask::operator bitset<size()>() const;
```

or alternatively through a named method which makes it explicitly clear that the conversion is happening:

```
constexpr bitset<size()> basic_simd_mask::to_bitset() const;
```

The output bitset value will have the same size as the `simd_mask`, and every element in the bitset will have the same value as its respective `simd_mask` element.

§ 2.1. Implementation experience

When working with ABIs which already have compact bit representations (e.g., AVX512 predicate registers) then converting to and from bitset is efficient. Conversion to and from wide element representations of masks (e.g., SSE or AVX), is more expensive but the `std::simd` library implementation is able to exploit the internal implementation details to make it more efficient than anything that the user could do using the public `std::simd` API.

§ 3. Conversion to and from integral value bit representations

There are several ways that `simd_mask`-like values could be stored, ranging from wide-element values (e.g., SSE, AVX), compact mask (e.g., ARM or AVX-512 predicates), bitsets, or byte-valued memory regions (e.g., `std::simd_mask::copy_to`). The `std::simd` API already has the ability to convert wide elements to and from `simd_masks` using, for example, the `operator+`. However, it can be useful to be able to convert to and from compact masks represented using raw bits stored in an integral value (something which `std::bitset` also supports explicitly). In this section we propose ways of constructing and accessing packed bits stored in integral values.

§ 3.1. Building a mask from a compact bit representation

When working with SIMD values of fixed sizes (rather than native types whose size can vary by target) it can be useful to express a mask pattern directly. For example, suppose a programmer requires a custom bit pattern, such as `0b10101101011010010101`. There is currently no easy or direct way to encode that pattern into a `basic_simd_mask` value.

We propose that the following constructor could be provided:

```
constexpr basic_simd_mask(auto unsigned_integral bits) noexcept;
```

The `simd_mask` is constructed such that the first (rightmost, least significant) M bit positions are set to the corresponding bit values of `bits`, where M is the smaller of N and the number of bits in the value representation of `pattern`. If M is less than the size of the mask then the remaining bit positions are initialized to zeroes.

The issue with using a `unsigned_integral` as the container for the input is that it might contain too many bits for a small mask, or too few bits for a big mask. Without a way of representing an arbitrary number of bits in an integral value (e.g., C23's bit-precise `_BitInt`) we are limited to only allowing up to 64-bits to be inserted into a `basic_simd_mask`, analagous to the same limits as a `std::bitset`.

§ 3.2. Compact bit accessor

There is currently no easy way for mask bits to be extracted from a `basic_simd_mask` in a compact form. Neither of the existing methods to extract is efficient when used for this. For example, a mask's values can be extracted as `bool`-like values by converting it to a `basic_simd` containing 8-bit elements.

```
auto t = simd_mask<uint8_t, 8>(m);  
auto asByteSimd = +t; // simd of bool-like objects
```

In this example the mask contents are converted to elements which are either 0 or 1 according to their respective mask bit. Even in this form there is still no easy step to convert to a compact bit representation.

To make it possible to extract compacted bits as an unsigned integral value we propose to borrow an idea from `bitset` and provide:

```
constexpr unsigned long long basic_simd_mask::to_ullong() const noexcept;
```

This will copy up to 64 mask elements to an output value, storing each mask element in a single bit. Unfortunately this potentially loses bits, since `to_ullong` will only emit those bits which can be contained in a 64-bit representation. As with the `unsigned_integral` constructor this could be solved if the C23 bit-precise `_BitInt` type was available in C++.

§ 3.3. Implementation experience

In some problem domains, such as telecommunications, compact representations of bits as integers are very common and it is very important to be able to efficiently convert to and from this format. Providing an API to make it easy to convert masks to and from this representation proved invaluable in writing concise meaningful code in this problem domain.

On a machine with compact mask representation (e.g., we tested on AVX-512) the masks are already stored in compact form, so converting to and from an integer representation was trivial.

On a machine with wide mask representation (e.g., SSE, AVX, AVX2) it is not easy or efficient to use compact representation if only the standard `std::simd` API can be used (e.g., converting to a byte memory and then from that to individual bits). Efficient conversion is only possible if target-specific instructions are used, and the programmer writes non-portable code to use them. For these targets then, it was better that a uniform API into a compact format was provided and handled efficiently by `std::simd` itself.

Constructing and accessing bits through the `bitset` pathway (e.g., `mask.to_bitset().to_ullong()`) proved also to be inefficient under some conditions. Even on targets which already had compact mask representation, the extra step in storing data temporarily in a `bitset` added to the overhead of the conversion. The extra step was difficult to remove because data would be moved in/out of a bitmask in 64-bit blocks, and in/out of a `simd_mask` in blocks whose sizes were governed by the operation in progress. This mismatch in sizes made it difficult to smooth out the data flow across the conversion.

§ 4. Initialising from a list

In earlier revisions of this proposal an initialisation-list-like mechanism was described, to enable something like the following:

```
simd<float, 4> myLut = {2.f, 9.f, 23.f, 45.f};
```

Since the earlier revisions, papers [P3299R2] and [P1928R15] have added CTAD and support for range/span constructors. These can be used to achieve an effect similar to that of the previous example:

```
basic_simd myLut = std::array{2.f, 9.f, 23.f, 45.f};           // simd<float,
```

Although slightly more verbose, it does satisfy our original requirements without needing additional features in the `simd` proposal, so we have removed the initialiser-list mechanism.

§ 5. Wording

The following wording is a diff against the current draft standard.

§ 5.1. Modify [**simd.mask.overview**]

In the header `<simd>` overview - [`simd.mask.overview`] - add at the end of the existing list of constructors.

```
// [simd.mask.ctor], basic_simd_mask constructors
constexpr explicit basic_simd_mask(value_type) noexcept;
template<size_t UBytes, class UAbi>
constexpr explicit basic_simd_mask(const basic_simd_mask<UBytes, UAbi>&)
template<class G> constexpr explicit basic_simd_mask(G&& gen) noexcept;
constexpr basic_simd_mask(const bitset<size()>& b) noexcept;
constexpr basic_simd_mask(auto unsigned_integral bits) noexcept;
```

Add after the conversion operators:

```
// [simd.mask.conv], basic_simd_mask conversion operators
template<class U, class A>
constexpr explicit(sizeof(U) != Bytes) operator basic_simd<U, A>() const
constexpr operator bitset<size()>() const noexcept;

// [simd.mask.namedconv], basic_simd_mask named type convertors
constexpr bitset<size()> to_bitset() const noexcept;
constexpr unsigned long long to_ullong() const noexcept;
```

§ 5.2. Modify `[simd.mask.ctor]`

At the end of the `[simd.mask.ctor]` section add two new constructors.

```
constexpr basic_simd_mask(const bitset<size()>& b) noexcept;
```

Effects:

Initializes the i^{th} element with `b[i]` for all `i` in the range `[0, size())`.

```
constexpr basic_simd_mask(auto unsigned_integral val) noexcept;
```

Effects:

Initializes the first `M` elements to the corresponding bit values in `val`, where `M` is the smaller of `size()` and the number of bits in the value representation (`[basic.types.general]`) of the type of `val`. If `M < size()`, the remaining bit positions are initialized to zero.

§ 5.3. Modify [simd.mask.conv]

At the end of the [simd.mask.conv] section add one new conversion operator.

```
constexpr operator bitset<size()>() const noexcept;
```

Returns:

A `bitset<size()>` object where the i^{th} element is initialized to `operator[](i)` for all i in the range `[0, size())`.

§ 5.4. Modify [simd.mask.namedconv]

After the [simd.mask.conv] add a new section called "`basic_simd_mask` named conversion operators".

`basic_simd_mask` named conversion operators [simd.mask.namedconv]

```
constexpr bitset<size()> to_bitset() const noexcept;
```

Preconditions:

- `size() <= N`, or
- For all i in the range `[N, size())`, `operator[](i)` is false.

Effects:

Equivalent to: `return operator bitset<size()>();`

```
constexpr unsigned long long to_ullong() const noexcept;
```

Let N be the number of bits in `unsigned long long`.

Returns:

The integral value corresponding to the bits in `*this`.

§ 6. Revision History

R1 => R2

- Removed initialiser list from the proposal.
- Added wording

R0 => R1

- Moved range constructor into its own paper [P3299R0]
- Updated discussion of initialiser-list-like constructor.

§ References

§ Informative References

[P1928R3]

Matthias Kretz. *Merge data-parallel types from the Parallelism TS 2*. 3 February 2023. URL: <https://wg21.link/p1928r3>

[P2638R0]

Daniel Towner. *Intel's response to P1915R0 for std::simd parallelism in TS 2*. 22 September 2022. URL: <https://wg21.link/p2638r0>

[P3299R0]

Daniel Towner, Matthias Kretz. *Range constructors for std::simd*. 22 May 2024. URL: <https://wg21.link/p3299r0>